

Automotive QA Intelligence Portal

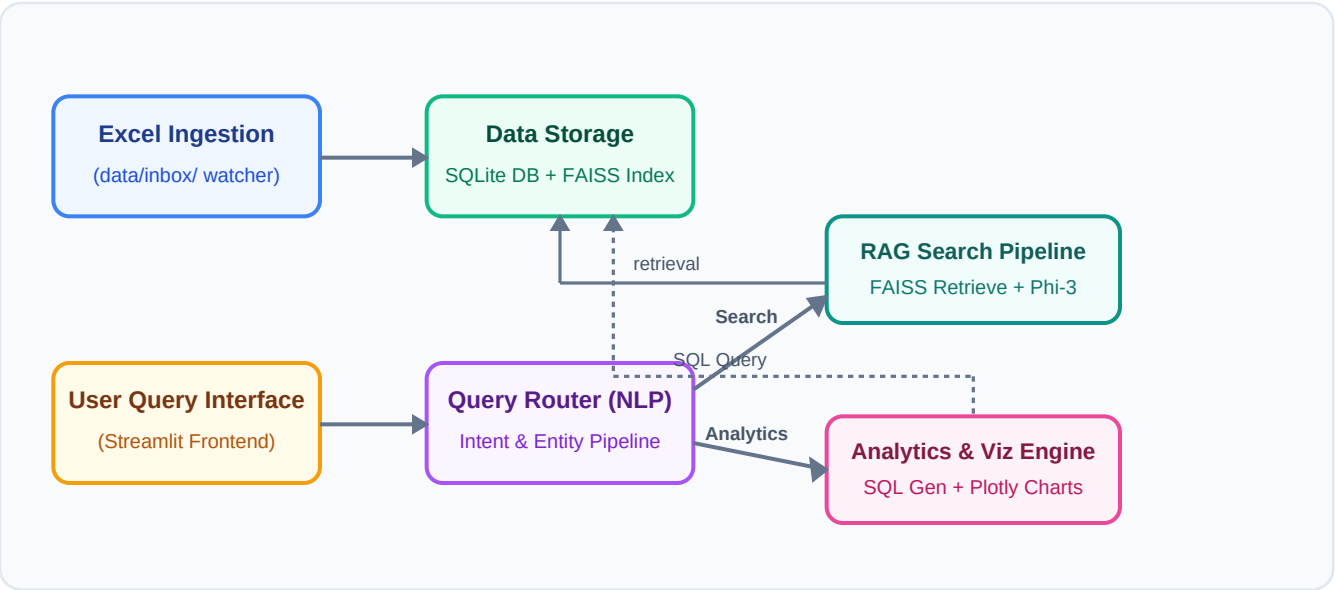
System Architecture, Working, & Design Specification Documentation

1. System Overview

The **Automotive QA Intelligence Portal** is an enterprise-grade, offline analytics and hybrid retrieval QA assistant system. It is designed to assist automotive technician operators and quality managers in analyzing field complaints (FTIR reports), tracking diagnostic trouble codes (DTCs), visualizing trends, and running secure, localized natural language retrieval using a local Large Language Model (Phi-3) and a FAISS vector database. The entire platform operates in a zero-network, high-security offline environment.

2. System Architecture & Flowchart

The architecture consists of three primary pipelines: Ingestion/ETL, Query Routing/NLP, and Execution (Search/RAG vs. Analytics/Visualization). The diagram below illustrates how data and user queries flow through the system:



3. Core Subsystems & Components

The following table details the key modules composing the portal, their responsibilities, and technologies:

Subsystem / Module	Primary Responsibility	Core Technologies
Ingestion Watcher (etl/watcher.py)	Monitors folder in background; reads new FTIR Excel sheets (.xlsx), validates structure, and updates database.	Python watchdog, pandas, openpyxl

Core Storage (data/)	Houses the structured relational database and indexed vector embeddings for retrieval.	SQLite, FAISS-CPU index, SentenceTransformers
Query Router (core/router.py)	Parses operator queries via NLP; classifies intent (e.g., Search, Analytics, Visualize) and extracts entities (Models, DTCs).	Spacy NLP, Regex pipelines
RAG Assistant (rag/)	Retrieves semantic matches from FAISS; synthesizes precise diagnostic suggestions offline using the local LLM.	HuggingFace Phi-3 (CPU/GPU local), FAISS
Analytics Engine (analytics/)	Executes structured SQL analytics on complaint records; supports multi-model comparison, DTC rate counting.	SQLite SQL compilation, pandas
Visualization (viz/charts.py)	Builds charts (histograms, donut charts, radar comparisons, trend lines) using enterprise styling themes.	Plotly Express, Plotly Graph Objects

4. Ingestion & Database Schema

The system starts by launching a daemon folder watcher thread monitoring the `data/inbox` directory. When a new Excel sheet is uploaded or saved, the watch-dog triggers the ETL process. The record is parsed, semantic fields are extracted (e.g., Subject, Complaint Description, Reported Company, Model Code, DTC), and coordinates are saved into the SQLite database. Concurrently, the embedding generator translates the text into a 384-dimensional dense vector and writes it to the local FAISS index.

Core SQLite Schema (Partial):

```
CREATE TABLE records (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  ftir_no TEXT UNIQUE NOT NULL,  
  subject TEXT,  
  reported_company TEXT,  
  outbreak_country TEXT,  
  product_model_code TEXT,  
  trouble_code_complaint TEXT,  
  summary TEXT,  
  report_year INTEGER,  
  report_month INTEGER  
);
```

5. Query Classification & Routing

When an operator submits a natural language question (e.g. 'Show the trend of DTC B1200 in 2025'), the **QueryRouter** executes a rule-and-regex-based classification pipeline. It labels the query with an intent class. The intent determines the execution path and the output layout of the Streamlit application.

Intent Name	Description / Pattern Match	Execution Path
SEARCH	Requests basic information or occurrences (e.g., 'Find all cases of transmission in Canada')	Executes RAG semantic matching, returns list of sources and details
ANALYTICS	Requests statistical queries, aggregate numbers, or group counts (e.g., 'How many cases of engine failure')	Runs aggregate SQL queries on SQLite database, displays table statistics
VISUALIZE	Requests visual charting of trends or counts (e.g., 'Plot line chart of sensor failures')	Generates aggregated dataset and maps to a styled Plotly Chart (Line, Bar, Radar, Donut)
COMPARE	Requests side-by-side analysis of multiple models or codes (e.g., 'Compare model AX and BY')	Executes pivot calculations or radar charts comparing relative failure profiles
REPORT	Requests document compilation (e.g., 'Generate monthly report for December 2025')	Invokes ReportEngine to output professional PDF/DOCX downloads for users

6. Quality Analytics & Visualization Engine

The system's quality dashboard is backed by the **engine** module, which compiles natural language parameters into structured queries. Once query routing isolates the target product model and outbreak parameters, the SQLite database is queried. The resulting dataframes are passed to the **viz** module, which builds beautiful Plotly charts following a light-themed color palette: soft blues (Hex: #3b82f6), teals (#0d9488), and ambers (#d97706). This gives the operator immediate, highly-readable visual feedback on quality hotspots.